Fundamentos de Aprendizagem Automática

Class 06 — Decision Trees and k-Nearest Neighbours

Mário Antunes

Universidade de Aveiro

2026

Models Without a Formula

Where We Left Off

linear
a line (Class 03)

probabilistic
a distribution
(Class 04)

margin, network
kernel, layers
(Class 05)

Class 06: no formula at all: *rules* and *neighbours*

- Until now a model was **a formula with parameters** that the optimizer fitted
- Today two models that are **algorithms on the data**: a *tree* of questions, and the *nearest neighbours*

What We Do Today

- **Decision tree:** a sequence of yes/no questions about the features; a prediction is the answer at the end of one path. A *symbolist* learner of Project 1
- **k-nearest neighbours (KNN):** to predict a new case, find the k most similar known cases and let them vote. An *analogizer* learner of Project 1
- Both are **non-parametric**: the model grows with the data; neither needs gradients; neither needs a loss to descend
- Both are easy to explain to someone outside machine learning, and both fail in instructive ways

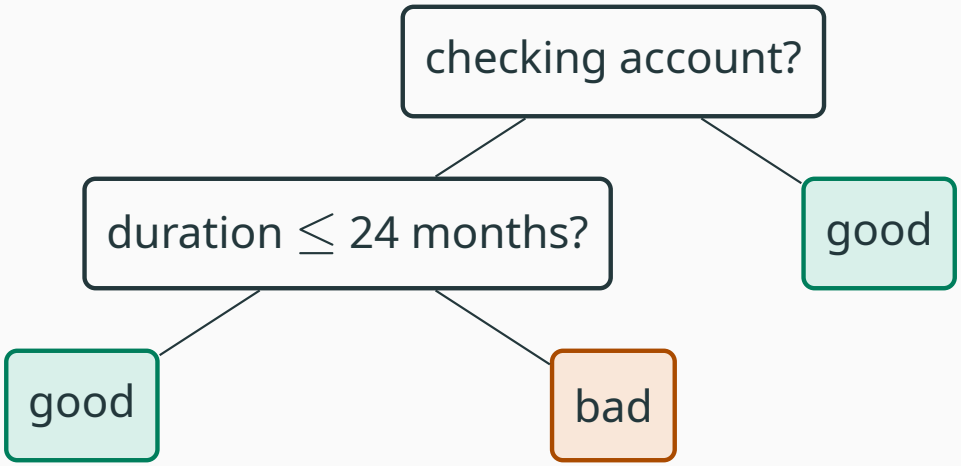
Roadmap of the Class

Part	Question	Answer
Decision trees	which question to ask first?	the one that makes the groups **purest**
	when to stop asking?	before the tree memorizes the noise
KNN	what is “similar”?	a **distance**, which needs scaled features
	how many neighbours?	the bias-variance knob, chosen on validation data
Choosing	which learner for my data?	compare on one protocol, with a test

Decision Trees

A Tree Is a Sequence of Questions

Everyone has used one: a flow chart. A bank deciding on a loan might ask:



- Each **internal node** asks a question about *one* feature; each **branch** is an answer
- Each **leaf** gives the prediction (a class, or a number)
- A new case follows one path from the root to a leaf: at most *depth* questions

A Tree Is a Set of Rules

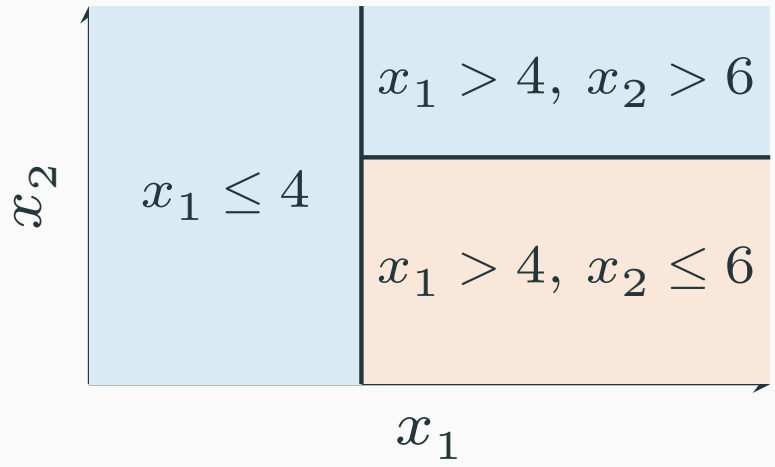
The tree of the breast-cancer lab (depth 3, trained on 426 tumours): each path is an `if ...` and `...` rule.

```
if worst_concave_points <= 0.147:
    if worst_radius <= 17.7:
     if worst_perimeter <= 108:  -> benign    (249 cases, 5 malignant)
      else:                        -> benign    (24 cases, 7 malignant)
    else:
     if worst_concavity <= 0.191: -> benign   (5 cases, 2 malignant)
      else:                       -> malignant  (16 cases, all malignant)
else: ... -> malignant (131 cases, 2 benign)
```

- A human can **read, check and challenge** every rule: the main reason to use a tree (medicine, credit, law)
- The counts in each leaf say how *sure* the rule is: a leaf with 1 case is a rule built on one patient

A Tree Cuts the Space into Boxes

With two features, each question cuts the plane with a vertical or a horizontal line: the leaves are **rectangles**, each with one prediction.



- Boundaries are **axis-parallel**: a diagonal line needs many small steps (a weakness)
- Unlike a linear model, one feature can matter in one region and not in another (*interactions* for free)

Which Question First?

A good question makes the groups it creates **purer**: more cases of one class in each. Example, 80 examples (40 and 40), two candidate questions:

question	left group	right group	how pure?
A	30 yes, 10 no	10 yes, 30 no	fairly
B	40 yes, 10 no	0 yes, 30 no	one group **perfectly** pure

- We want a **number** that measures the impurity of a group, and the *gain* of a question
- Then "which question first?" becomes "the question with the largest gain": try them all

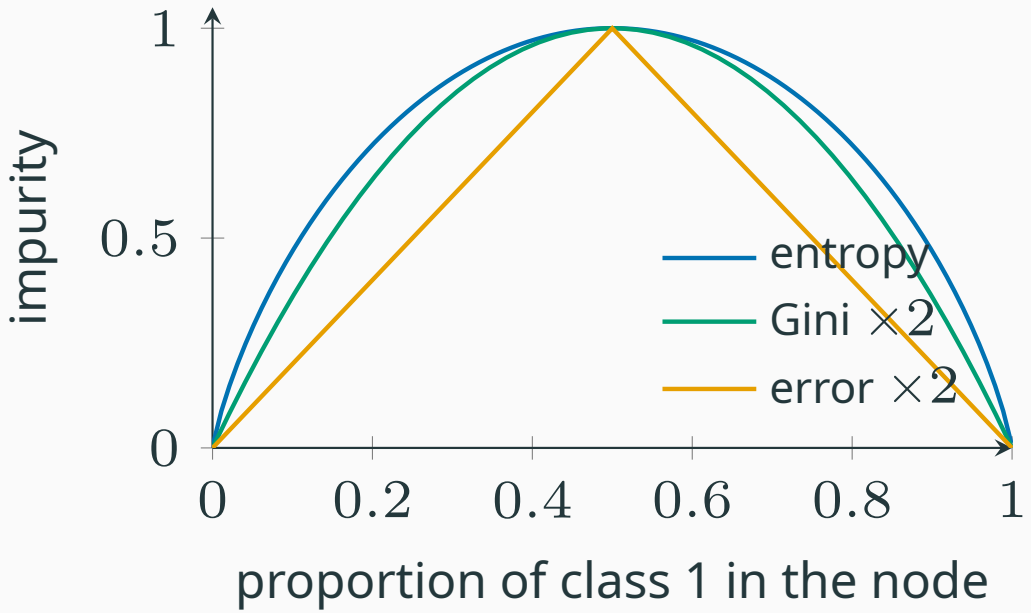
Measures of Impurity

For a group with class proportions $p_1, \ldots, p_K$ (proportions of the group that belong to each class):

- **Gini index:** $G = 1 - \sum_k p_k^2$ — the probability that two examples drawn at random from the group have different classes
- **Entropy:** $H = -\sum_k p_k \log_2 p_k$ — the average number of bits needed to name the class (Class 01)
- **Error:** $E = 1 - \max_k p_k$ — the share of the group that the majority vote gets wrong
- All are 0 for a pure group and largest when the classes are equally likely

group	p	Gini	entropy	error
9 yes, 5 no	$(0.64, 0.36)$	0.459	0.940	0.357
5 yes, 5 no	$(0.5, 0.5)$	0.500	1.000	0.500
10 yes, 0 no	$(1, 0)$	0	0	0

The Three Curves



- Entropy and Gini are **smooth and strictly curved**: a split that makes both children a little purer *always* lowers them
- The error is a **straight line** on each side: many useful splits leave it unchanged (next slides)

The Gain of a Question

A question splits a group of n examples into a left part of n_L and a right part of n_R. Its **gain** is the impurity before minus the *weighted average* impurity after:

$$\text{gain} = I(\text{parent}) - \frac{n_L}{n} I(\text{left}) - \frac{n_R}{n} I(\text{right})$$

- The weights n_L/n and n_R/n make a big group count more than a small one
- The gain is never negative: asking never makes the average group *less* pure (impurity is a concave function)
- **Example** (the 80 examples above, Gini): parent $= 0.5$. Question B: left $(40, 10)$ has $G = 0.32$, right $(0, 30)$ has $G = 0$; gain $= 0.5 - \frac{50}{80} \cdot 0.32 - \frac{30}{80} \cdot 0 =$ **0.30**
- Question A: both children $(30, 10)$ and $(10, 30)$ have $G = 0.375$; gain $= 0.5 - 0.375 = 0.125$

Worked Example: Play Tennis? (1/2)

14 days: is the weather good enough to play? 9 days yes, 5 days no. The root question must be chosen among four features.

feature	value	yes	no	entropy of the group
outlook	sunny	2	3	0.971
	overcast	4	0	**0**
	rain	3	2	0.971
humidity	high	3	4	0.985
	normal	6	1	0.592

- Parent: $H = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940$
- The *overcast* group is **pure**: always play

Worked Example: Play Tennis? (2/2)

Gain of **outlook**: the weighted entropy after the question, and the difference:

$$\frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971) = 0.694$$

$$\text{gain} = 0.940 - 0.694 = \mathbf{0.247}$$

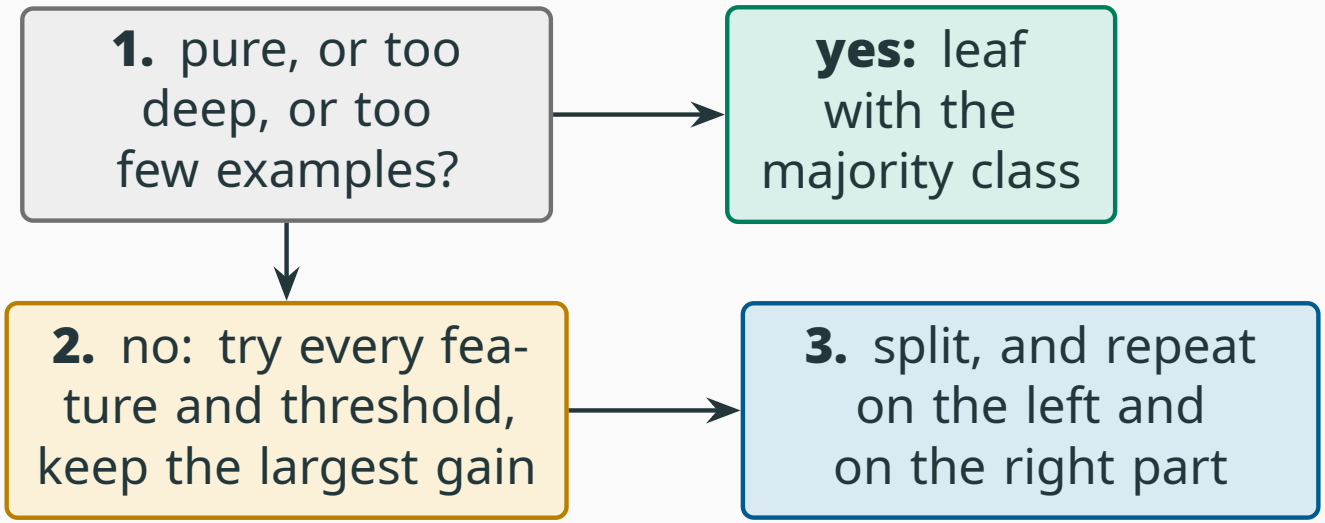
question	gain (entropy)	gain (Gini)	gain (error)
outlook	**0.247**	**0.116**	0.071
humidity	0.152	0.092	0.071
windy	0.048	0.031	0.000
temperature	0.029	0.019	0.000

- Outlook is the root under all three criteria; the *error* gives **zero** gain to two useful questions (demo T1)

Entropy or Gini?

- In practice they choose the **same** question almost always: the two curves have the same shape
- Entropy is slightly more sensitive to *small* classes; Gini is cheaper (no logarithm)
- Breast cancer, best question: `worst_concave_points`, threshold 0.147 for both criteria (lab Part B)
- Default in scikit-learn: Gini. C4.5 (Quinlan) uses a normalized entropy gain; CART (Breiman) uses Gini
- **Misclassification error** is a bad *splitting* criterion (it is flat across useful splits) but is the right measure of the *final* tree

The Greedy Algorithm



- **Greedy:** each split is the best *now*, without looking ahead; the best *tree* is a hard (NP-complete) search
- A very good question may come after a mediocre one (XOR!): greedy can miss it
- The same algorithm works for classes and for numbers: only the impurity changes

Numeric Features: Where to Cut?

For a numeric feature the candidate questions are "$x \leq t$", with t the **midpoint between consecutive sorted values** (any other threshold gives the same partition). Six applicants, bad credit $= 1$:

age	22	25	31	38	45	52
bad?	1	1	0	1	0	0

threshold	left (good, bad)	right (good, bad)	gain (Gini)
23.5	(0, 1)	(3, 2)	0.100
28.0	(0, 2)	(3, 1)	**0.250**
34.5	(1, 2)	(2, 1)	0.056
41.5	(1, 3)	(2, 0)	**0.250**
48.5	(2, 3)	(1, 0)	0.100

One Sorted Scan Finds the Best Cut

Sort the values once; walk along them keeping the **cumulative class counts** on the left; the right counts are the totals minus the left. The gain of every threshold is then a few operations.

- Cost: one sort, $O(n \log n)$, then a linear pass; for all d features $O(d\, n \log n)$ per node
- Without the cumulative counts, each of the $n - 1$ thresholds would need a pass over the data: $O(n^2)$
- The example above has **two thresholds tied** at 0.250 (28 and 41.5): the algorithm takes one of them; another implementation may take the other. This is why two correct trees can differ
- Breast cancer, `worst_perimeter`: 513 candidate thresholds, the best at 106.0

Categorical Features and Missing Values

- **Categorical feature:** a question "is the value in this subset?"; with one-hot columns (d yes/no questions) the tree asks about one category at a time
- A feature with **many categories** (a postcode) gives many ways to split: a high risk of fitting noise (see importances)
- **Missing values:** CART uses *surrogate splits*; scikit-learn trees accept them natively since version 1.3; or impute (Class 03)
- **Scaling is not needed:** a tree only compares a feature with a threshold; multiplying a feature by 1000 changes nothing
- Monotone transformations (logarithm) change nothing either

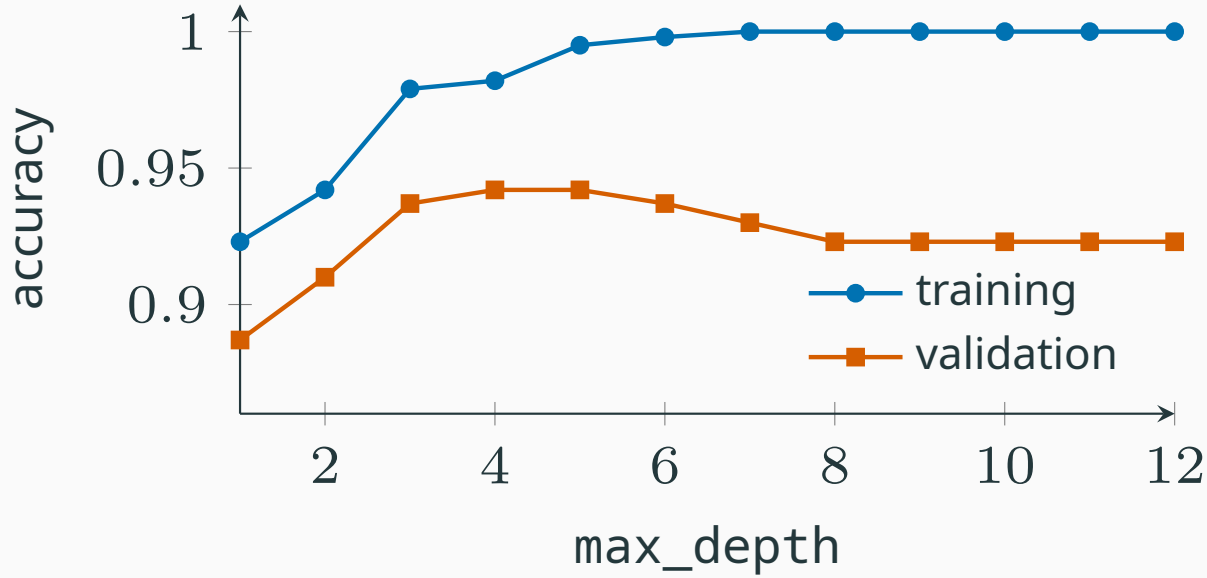
When to Stop

A tree grown until every leaf is pure has **100% accuracy on the training data** and has memorized the noise. Stopping rules:

- The node is **pure** (one class): nothing to gain
- **Maximum depth** reached (`max_depth`): at most that many questions
- **Minimum leaf size** (`min_samples_leaf`): a leaf must have enough examples to be trusted
- **Minimum gain** (`min_impurity_decrease`): the best question is not worth asking
- These are *hyperparameters*: chosen on **validation data**, never on the training accuracy (Class 01)

Depth and Overfitting

Breast cancer, accuracy against the maximum depth (training data, and 10-fold cross-validation):



- Training accuracy reaches 1.000 at depth 7; the validation accuracy peaks at **depth 4-5 (0.942)** and then falls to 0.923

Depth Depends on the Data

Best depth by 10-fold cross-validation, and the accuracy of an unlimited tree (depth 15):

data set	best depth	CV accuracy at best	train, depth 15	CV, depth 15
breast cancer	5	0.942	1.000	0.923
German credit	3	0.717	1.000	0.685
wine (≥ 6)	15	0.780	0.978	0.780

- A small, noisy set (German credit) wants a **shallow** tree; a large set with a real structure (6 497 wines) can use all its depth
- The gap training — validation is the symptom: 0.315 for German credit at depth 15

Pre-Pruning: the Leaf Size

`min_samples_leaf`: a leaf with fewer examples is not allowed. 10-fold CV accuracy:

data set	criterion	leaf ≥ 1	≥ 5	≥ 20	≥ 50
breast cancer	Gini	0.923	0.933	0.917	0.886
	entropy	0.931	**0.942**	0.931	0.875
German credit	Gini	0.679	0.704	0.711	**0.713**
	entropy	0.703	0.687	0.713	**0.724**
wine (≥ 6)	entropy	**0.794**	0.761	0.750	0.743

- The right size depends on the data; noisy data want large leaves; the criterion matters little

Post-Pruning: Cost-Complexity

Grow the full tree, then cut back. Penalize the **number of leaves** $|T|$ as ridge penalizes the weights (Class 03):

$$R_\alpha(T) = \underbrace{\text{error}(T)}_{\text{on the training data}} + \alpha\,|T|$$

- $\alpha = 0$: the full tree; a larger α removes the branch that pays *least* per leaf, then the next: a nested **path** of trees from one fit
- German credit, 700 training examples, validation accuracy along the path:

leaves	142 (full)	96	63	41	**28**	16	8
train accuracy	1.000	0.954	0.909	0.874	0.836	0.787	0.763
validation accuracy	0.700	0.727	0.727	0.727	**0.753**	0.743	0.690

Regression Trees

For a **number** the tree is the same: the leaf predicts the **mean** of its examples, and a question is judged by how much it reduces the **variance** (the sum of squared errors, SSE, around the mean):

$$\mathrm{SSE}(S) = \sum_{i \in S} (y_i - \bar{y}_S)^2$$

$$\text{gain} = \mathrm{SSE}\big(\text{parent}\big) - \mathrm{SSE}\big(\text{left}\big) - \mathrm{SSE}\big(\text{right}\big)$$

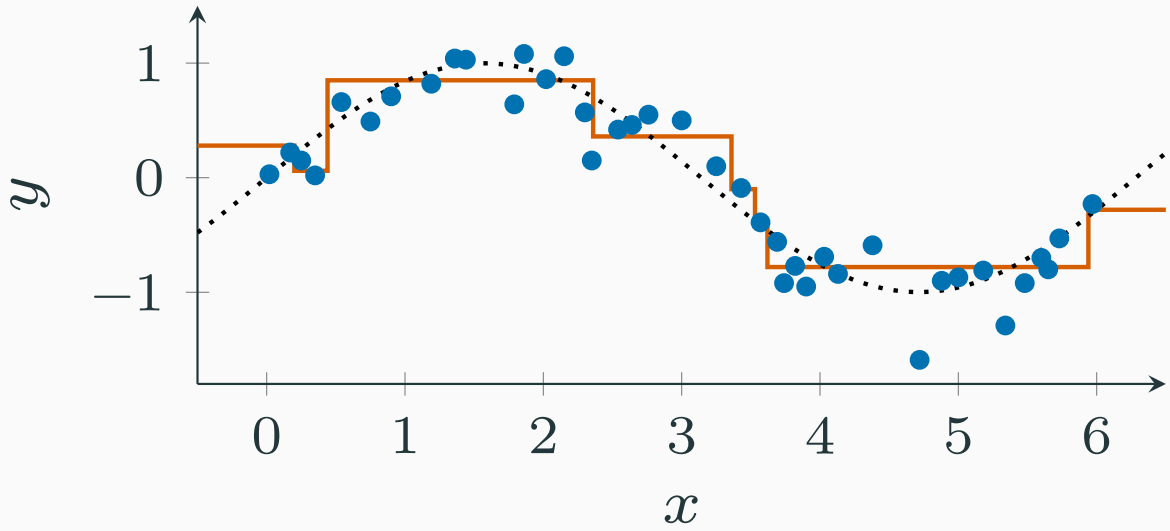
- The tree is grown exactly as before: all features, all thresholds, the largest gain, recurse

A Regression Split by Hand

Five points (x, y): $(1, 1.0), (2, 1.2), (3, 3.1), (4, 3.0), (5, 3.2)$; without a split the mean is 2.30 and the SSE 4.84.

threshold	left mean	right mean	total SSE
1.5	1.00	2.63	2.728
2.5	1.10	3.10	**0.040**
3.5	1.77	3.10	2.707
4.5	2.08	3.20	3.828

The Prediction Is a Staircase



- Depth 3: **8 leaves**, 8 different values; outside $[0, 6]$ the tree repeats its first or last value

A Tree Cannot Extrapolate

Train California housing on the 70% of districts with the lowest median income; test on the **richest 30%**, never seen:

model	RMSE, seen range	RMSE, richest 30%
mean	0.888	1.780
linear regression	0.637	**0.755**
tree, depth 6	0.610	1.464
tree, leaves ≥ 20	0.466	1.211

- Inside the data the tree is better than the line (0.466 vs 0.637); outside it is **much worse** (1.21 vs 0.76): it predicts a constant, the line continues the trend
- Same for KNN (next section): both **interpolate**; only models with a global form extrapolate

Regression Trees on Real Data

5-fold RMSE (concrete in MPa, abalone in rings, auto-mpg, California in 100 000 USD):

data set	mean	linear	tree $d = 3$	tree $d = 6$	tree $d = 12$	leaf ≥ 10
concrete	16.70	10.43	10.55	7.71	**6.75**	7.21
abalone	3.22	**2.21**	2.50	2.35	2.80	2.39
auto-mpg	7.79	3.39	3.58	3.65	3.83	**3.18**
California	1.15	0.67	0.80	0.71	0.73	**0.65**

- Concrete: strong **interactions** (cement with age): the tree wins by a wide margin; abalone is nearly linear: the line wins
- Depth 12 is the best on concrete and the worst on abalone: **choose on validation data**

Strengths and Weaknesses

strengths	weaknesses
readable rules, easy to explain	high **variance**: small data change, other tree
no scaling, no encoding of order	axis-parallel boundaries: diagonals are staircases
numeric and categorical features	cannot extrapolate (regression)
captures interactions and non-linearity	greedy: not the best tree
fast to train and to predict ($\leq$ depth questions)	biased importances (next slide)

- The weakness that matters most is the **variance**: the topic of the next class (ensembles: *bagging*, *random forests*, *boosting*)

Instability

Breast cancer; 30 trees, each trained on a **bootstrap sample** (draw n cases *with* replacement):

feature at the root	worst perimeter	worst concave points	worst radius	worst area	mean concave points
number of trees (of 30)	19	6	2	2	1

- The test accuracy of the 30 trees: mean 0.915, **sd 0.017**, from 0.871 to 0.947; between 6 and 18 leaves
- The *majority vote* of the 30 trees: 0.918 — a first look at the idea of **bagging**
- Moral: a single tree is a fragile object; do not read too much into *the* first split

Feature Importances and Their Bias

The *impurity importance* of a feature is the total gain of its splits. But it is **biased**: wine data plus one column of *pure noise*:

feature	impurity importance (train)	permutation importance (test)
alcohol	0.261	0.127
volatile acidity	0.152	0.077
sulphates	0.091	0.036
random continuous column	**0.056**	-0.001
random binary column	0.003	0.001

- The noise column gets 5.6% (many thresholds, many chances to fit noise); **permutation importance** (shuffle one column on held-out data) sees through it
- Two copies of a feature **split** the credit: 0.261 becomes $0.027 + 0.234$

DEMO T1–T3:

`01_decision_trees.ipynb`

DEMO T4–T5:

`01_decision_trees.ipynb`

k-Nearest Neighbours

Learning by Analogy

How does a doctor, or a real-estate agent, estimate? “Cases like this one were …”. KNN does exactly that.

- **No training:** the “model” is the stored data set; learning is *lazy*
- To predict a new case: find the k **most similar** stored cases and combine their answers: a **vote** for a class, the **mean** for a number
- Two choices define the learner: the **distance** (what is “similar”) and k (how many)
- Very old idea (Fix and Hodges, 1951; Cover and Hart, 1967); still a strong baseline
- An *analogizer* in the five classes of learners of Project 1 (with the SVM of Class 05)

The Algorithm in Four Steps

1. **Standardize** the features (training statistics only; Class 03)
2. **Distance:** compute the distance of the new case to every training case
3. **Neighbours:** keep the k smallest distances
4. **Combine:** majority vote of their classes (or the mean of their targets)

- Nothing is fitted; all the cost is paid **at prediction time**: $O(n\,d)$ per case
- The decision boundary is never written down; it is *implied* by the data
- $k = 1$: the prediction is the class of the single nearest case

Worked Example: Six Cases, One Query

Query $q = (2.6, 1.2)$; the data and their distances $\sqrt{(x_1 - 2.6)^2 + (x_2 - 1.2)^2}$:

case	position	class	distance to q	rank
C	$(3, 1)$	1	0.447	1
A	$(1, 1)$	0	1.612	2
D	$(3, 3)$	1	1.844	3
B	$(1, 3)$	0	2.408	4
F	$(6, 4)$	0	4.405	5
E	$(5, 5)$	1	4.494	6

- $k = 1$: C $\rightarrow$ class **1**. $k = 3$: C, A, D: votes $1 : 2, 0 : 1$ $\rightarrow$ class **1**
- $k = 5$: C, A, D, B, F: votes $1 : 2, 0 : 3 \rightarrow$ class **0**: the answer **changed** with k

Distances

For two cases $a = (1, 2)$ and $b = (4, 6)$:

distance	formula	value
Euclidean	$\sqrt{\sum_j (a_j - b_j)^2}$	$\sqrt{9 + 16} = 5$
Manhattan	$\sum_j \lvert a_j - b_j \rvert$	$3 + 4 = 7$
Chebyshev	$\max_j \lvert a_j - b_j \rvert$	4
cosine	$1 - \frac{a^\top b}{\lVert a \rVert \, \lVert b \rVert}$	$1 - \frac{16}{2.24 \cdot 7.21} = 0.008$
Hamming	number of differing coordinates	2

- Euclidean is the default; Manhattan is less sensitive to one large difference; cosine ignores the *length* (text); Hamming for categories

Which Distance? (Demo K1)

Test accuracy of KNN, $k = 5$, standardized features:

data set	Euclidean	Manhattan	Chebyshev	cosine
breast cancer	0.951	**0.958**	0.944	0.930
digits	**0.973**	0.958	0.938	0.960
spam	0.932	**0.938**	0.912	0.933
heart disease	0.803	0.803	0.750	**0.829**

- No distance wins everywhere; differences of 1-3 points are within the noise of one split
- The distance is a **modelling decision**: it says what "similar" means for *your* problem

Why Scaling Is Essential

Two patients: a = (age 30, income 30 000), b = (age 40, income 30 500). They differ by 10 years and by 500 euros.

- Raw squared distance: $10^2 + 500^2 = 100 + 250\,000$: the **income decides**, the age counts 0.04%
- Standardized (sd of age 10, of income 10 000): $1^2 + 0.05^2 = 1.0025$: now the **age decides**
- Neither is "right": scaling gives each feature the **same chance**, which is a statement about the problem
- The rule of Class 03: statistics from the **training** part only

Scaling in Numbers (Demo K2)

Test accuracy, $k = 5$, raw against standardized features:

data set	raw	standardized	range ratio (largest / smallest)
breast cancer	0.916	**0.951**	140 569
wine (≥ 6)	0.676	**0.763**	8 367
German credit	0.616	**0.692**	18 174
heart disease	0.618	**0.803**	438
digits	**0.980**	0.973	16
spam (log1p)	0.929	0.932	8

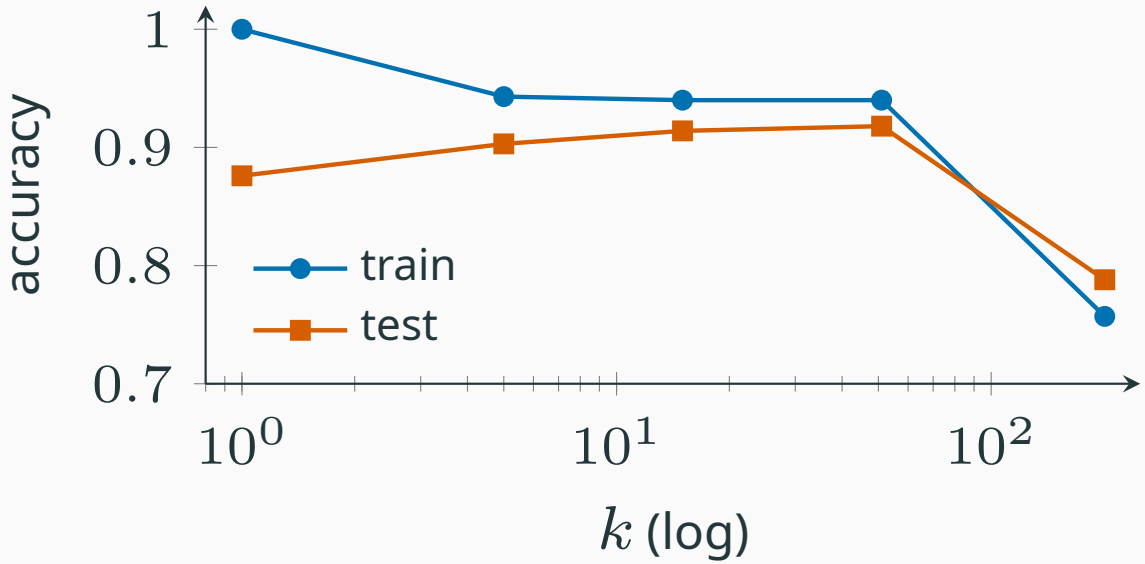
- The more different the ranges, the more standardization matters: up to **18 points** (heart disease)
- The digits are 64 pixels on the **same** scale: nothing to fix (and standardizing slightly hurts, as it amplifies rare pixels)

Choosing k: the Bias-Variance Knob

- $k = 1$**:** the boundary follows every single case, including the mislabelled ones: low bias, **high variance**, zero training error
- $k = n$**:** every prediction is the majority class: high bias
- In between: a smooth boundary; the best k depends on the noise and is chosen on **validation data**
- Rule of thumb: start with $k \approx \sqrt{n}$, use an odd k for two classes (no ties), search on a grid

two moons (300 train)	$k = 1$	5	15	51	201
training accuracy	1.000	0.943	0.940	0.940	0.757
test accuracy	0.876	0.903	0.914	**0.918**	0.788

The Curve of k



- Best k by 5-fold CV: **11** (breast cancer, 0.967), **5** (digits, 0.977), **9** (German credit, 0.735)
- German credit: $k = 1$ gives 0.677 and $k = 59$ gives 0.710 — both worse than $k = 9$; breast cancer and digits drop a little with a large k

Weighted Voting

Instead of one vote per neighbour, let a neighbour vote with weight $w_i = 1/d_i$: the near ones count more.

- Example from the six cases, $k = 5$: weights $1/0.447 = 2.24$ (C, class 1), 0.62 (A, 0), 0.54 (D, 1), 0.42 (B, 0), 0.23 (F, 0)
- Class 1: $2.24 + 0.54 = 2.78$; class 0: $0.62 + 0.42 + 0.23 = 1.26 \rightarrow$ class **1** (the plain vote gave 0)
- It makes ties rare and the result less sensitive to k: spam, CV accuracy for $k = 3, 15, 45$

spam	$k = 3$	$k = 15$	$k = 45$
uniform	0.927	0.926	0.917
distance-weighted	0.936	0.939	**0.937**

KNN for Regression

The prediction is the **mean** of the targets of the k nearest cases (or the distance-weighted mean): a *local average*.

- A noisy sinusoid, 80 points; error to the *true* curve inside the data: $k = 1$: 0.254, $k = 3$: 0.158, $k = 10$**:** **0.109**, $k = 40$: 0.278
- $k = 1$ passes through every point (training RMSE 0); $k = 40$ flattens the peaks
- Like a tree: **no extrapolation**; the prediction outside the data is the value of the nearest border
- Unlike a tree: the prediction is *smooth in the data*, with no axis-parallel steps

KNN Regression in Numbers (Demo K4)

5-fold RMSE, standardized features:

data set	mean	linear	$k = 1$	$k = 5$	$k = 15$	$k = 15$ weighted
concrete	16.70	10.43	9.61	8.71	9.79	**8.48**
abalone	3.22	**2.21**	2.91	2.29	2.22	2.21
auto-mpg	7.79	3.39	3.41	3.09	3.16	**3.03**
California	1.15	0.67	0.81	0.64	0.62	**0.62**

- On concrete and California a local model beats the line (they are non-linear); on abalone (nearly linear, noisy) the line is as good
- $k = 1$ is never the best: it copies the noise

The Curse of Dimensionality

In high dimension *everything is far from everything*. 500 random points in the unit cube $[0, 1]^d$ and one query:

d	1	2	10	50	100	1 000
nearest / farthest distance	0.002	0.012	0.31	0.63	0.74	**0.90**
edge of the cube holding 1% of the data	0.01	0.10	0.63	0.91	0.95	0.995

- At $d = 1000$ the nearest point is only 10% closer than the farthest: “nearest” has lost its meaning
- To capture 1% of the data in $d = 100$ you need a cube of edge **0.95**: a “neighbourhood” is nearly the whole space (Class 01)

Irrelevant Features

Add r columns of **pure noise** to the breast-cancer data (standardized). Test accuracy:

noise columns r	0	10	100	300	1 000
KNN, $k = 5$	0.947	0.918	0.877	0.889	**0.836**
logistic regression	0.959	0.953	0.942	0.918	0.895
SVM (RBF)	0.953	0.953	0.942	0.930	0.871
decision tree	0.912	0.930	0.906	0.901	0.906
naive Bayes	0.912	0.912	0.912	0.912	0.924

- KNN counts every column in the distance, so it degrades fastest; a **tree** ignores a column that never gives a gain; naive Bayes treats columns independently
- Remedy: select or weight features *before* KNN (the filter, wrapper, and embedded methods of feature selection)

The Cost of Prediction

Seconds to predict 500 queries, $k = 5$:

training cases n	dimension d	brute force	kd-tree	ball tree
2 000	2	0.004	0.003	0.011
20 000	2	0.030	**0.004**	0.012
20 000	10	**0.042**	0.145	0.205
20 000	30	**0.040**	0.519	0.319
20 000	100	**0.025**	2.118	1.707

- A **kd-tree** (a tree of coordinate cuts of the *data*) prunes most of the distances in low dimension; beyond $d \approx 10$ it is *slower* than brute force (the curse again)
- Memory: the whole data set; 10^6 cases $\times$ 100 features in float64 = 0.8 GB, kept for every prediction

Strengths and Weaknesses

strengths	weaknesses
no training; adds new data at once	slow, memory-hungry prediction
non-linear boundary, any shape	needs scaling; **sensitive to irrelevant features**
few hyperparameters (k, distance)	suffers in high dimension
easy to explain: "like these cases"	no model to read or to export
multi-class and regression for free	no extrapolation; noisy for small k

- A **baseline** to try early: a few lines, and a number that more complex models must beat
- A distance is also useful *outside* classification: clustering (Class 10), anomaly detection, recommendation

DEMO K1–K3:

`02_knn.ipynb`

DEMO K4–K5:

`02_knn.ipynb`

Choosing a Learner

The Five Classes of Learners, Revisited

class of learner	learns by	taught	what we built
Symbolists	rules, inverse deduction	06	decision tree
Connectionists	adjusting weights	03, 05	logistic regression, MLP
Evolutionaries	evolving candidates	02	genetic algorithm, DE
Bayesians	updating probabilities	04	naive Bayes, MAP
Analogizers	similarity to known cases	05, 06	SVM with a kernel, KNN

- Project 1: one learner **per class**, the same data, the same methodology; for the *evolutionaries* the evolution must be the way the model is **learned**
- Each class has its own idea of *what is learned*: rules, weights, populations, beliefs, similarities

One Protocol, Five Learners (Demo K5)

Median test accuracy, 5 stratified 75/25 splits, standardization on the training part, k and `min_samples_leaf` tuned by inner 5-fold CV:

data set	logistic	naive Bayes	SVM (RBF)	tree	KNN
breast cancer	**0.972**	0.923	0.965	0.923	0.965
digits	0.969	0.816	**0.982**	0.842	0.967
spam	0.942	0.842	**0.950**	0.912	0.932
wine (≥ 6)	0.730	0.677	0.774	0.764	**0.780**
German credit	0.744	0.696	**0.752**	0.692	0.732
heart disease	**0.842**	**0.842**	**0.842**	0.776	0.829

- The SVM wins or ties on four of six; KNN wins on wine; the plain logistic regression is never far

Is a Difference Real?

The corrected resampled t-test of Class 05, KNN against three learners (difference of the mean accuracy, p-value):

data set	KNN — tree	KNN — SVM	KNN — logistic
breast cancer	$+0.036$ (0.012)	-0.004 (0.51)	-0.008 (0.37)
digits	$+0.127$ (0.001)	-0.013 (0.063)	$+0.001$ (0.89)
spam	$+0.024$ (0.012)	-0.015 (0.035)	-0.008 (0.085)
wine	$+0.016$ (0.054)	$+0.011$ (0.24)	$+0.048$ (0.007)
German credit	$+0.030$ (0.31)	-0.031 (0.038)	-0.027 (0.21)

- KNN is clearly **better than a single tree** on four of five data sets; against the SVM and the logistic regression, differences are small or in their favour
- With 5 splits only large differences are detected: use more splits for close competitors

A Decision Guide

situation	first try
small tabular data, a baseline	logistic regression, KNN
rules must be explained to a person	decision tree (shallow)
non-linear, medium data (up to $\sim 10^4$)	SVM with RBF, KNN
many cases, high accuracy needed	network, ensembles (next class)
features on very different scales	any model, **after scaling** (not trees)
many irrelevant features	tree, regularized linear model, not KNN
must extrapolate	linear model, not tree or KNN

- Always: a **baseline**, a **split** that matches the problem, a **spread** over splits, a **test** for the claim

Key Takeaways (1/2)

- **Tree:** a sequence of questions; each chosen **greedily** to maximize the **gain** of a purity measure (Gini, entropy); numeric features are cut at midpoints found by a sorted scan
- **Overfitting:** an unlimited tree memorizes (training accuracy 1.000); limit the depth or the leaf size, or **prune** by cost-complexity, *on validation data*
- **Regression tree:** leaf means, variance reduction, a staircase that **cannot extrapolate**
- **Strengths:** readable rules, no scaling, interactions; **weakness:** high variance and biased importances (permutation importance is safer)

Key Takeaways (2/2)

- **KNN:** no training; the k nearest cases vote (or average); the **distance** and k define the learner
- **Scale** the features: otherwise the largest unit decides; k is the bias-variance knob
- **Curse of dimensionality:** irrelevant features and high d destroy the distance; prediction is $O(nd)$
- Both are **local** methods and do not extrapolate; both are good baselines
- **Choose** on one protocol, with a spread and a corrected test; the next class turns the weakness of the tree (variance) into a strength

Lab 06: Build It from Scratch

Part	Task	twin
A	Data visualization and evaluation methodology	—
B	Gini, entropy, the best threshold (one sorted scan)	`DecisionTreeClassifier`
C	A classification tree: recursion, prediction, rules	`DecisionTreeClassifier`
D	Depth on validation data, pruning	`ccp_alpha`
E	A regression tree: variance reduction	`DecisionTreeRegressor`
F	KNN with `jax.numpy`: vote, scaling, k, noise	`KNeighborsClassifier`
G	KNN for regression	`KNeighborsRegressor`
H	Challenge: the symbolist and the analogizer	`GridSearchCV`

- Guide `practice/06_trees_knn.pdf`; notebook `lab06_trees_knn.ipynb`; solution in `solutions/`
- **Project 1:** the tree is your *symbolist*, KNN or the SVM your *analogizer*

References (1/2)

- L. Breiman, J. Friedman, R. Olshen, C. Stone, *Classification and Regression Trees*, 1984
- J. R. Quinlan, “Induction of decision trees”, *Machine Learning*, 1986; *C4.5: Programs for Machine Learning*, 1993
- T. Cover, P. Hart, “Nearest neighbor pattern classification”, *IEEE Trans. Information Theory*, 1967
- T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, 2009: chapters 9.2 (trees), 13 (nearest neighbours)
- G. James et al., *An Introduction to Statistical Learning with Applications in Python*, 2023: chapters 8 (trees), 2.2.3 (k-NN)

References (2/2)

- C. Bishop, *Pattern Recognition and Machine Learning*, 2006: section 14.4 (tree-based models)
- C. Strobl, A.-L. Boulesteix, A. Zeileis, T. Hothorn, “Bias in random forest variable importance measures”, *BMC Bioinformatics*, 2007
- R. Beyer, J. Goldstein, R. Ramakrishnan, U. Shaft, “When is nearest neighbor meaningful?”, *ICDT*, 1999
- J. Bentley, “Multidimensional binary search trees used for associative searching”, *Communications of the ACM*, 1975
- P. Domingos, *The Master Algorithm*, 2015